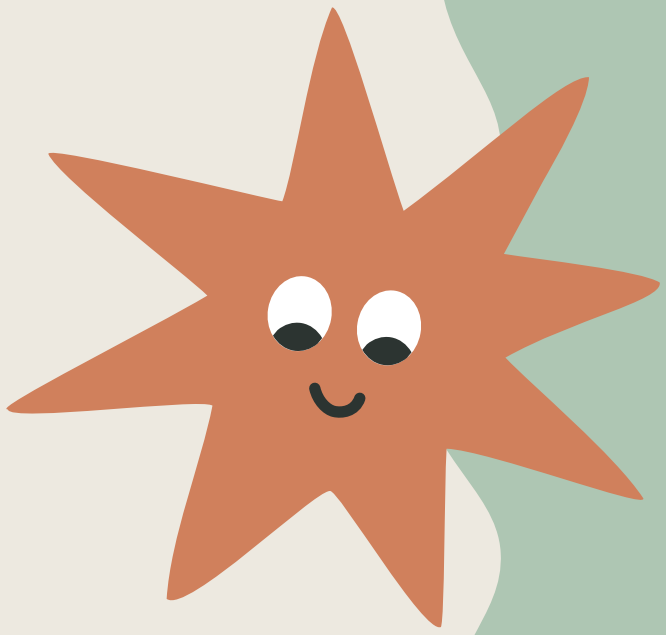
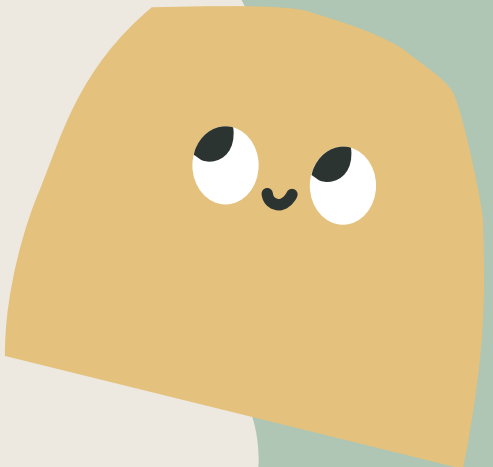
Implementation of Depth-First Search (DFS) in Python

Lecture 13





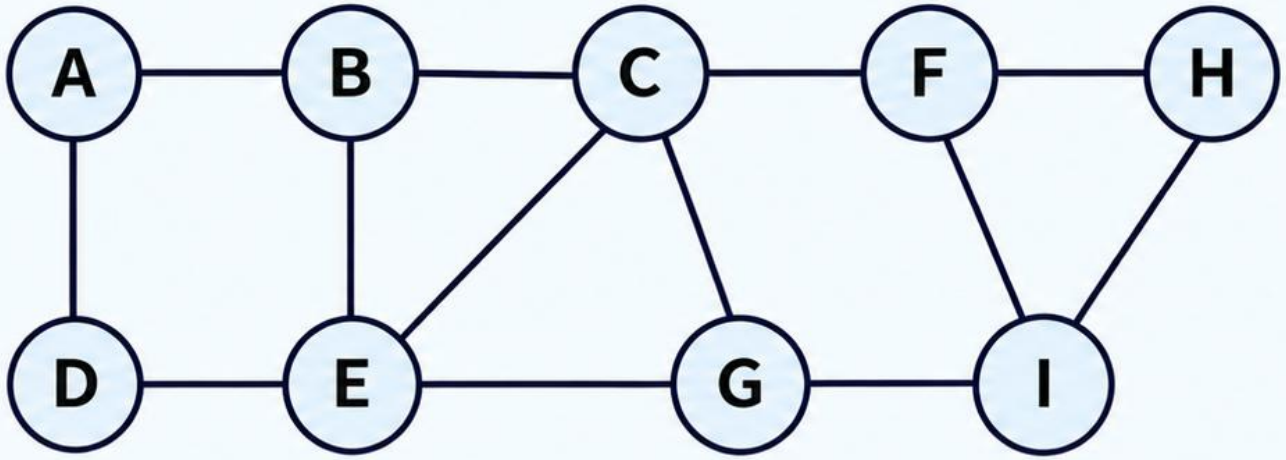
Learning Objectives

➢By the end of this lecture, students should be able to:

- Represent a graph in Python using an adjacency list.
- Implement DFS iteratively using a stack.
- Use a set to track visited nodes.
- Explain why nodes are marked visited when pushed onto the stack.
- Trace the stack and visited set during execution.
- Produce the DFS traversal order.
- Understand the time and space complexity of the implementation.

Prepared By: Ehtisham Rasheed

Graph Representation



Python Representation (Adjacency List)

```python
# Graph represented as an adjacency list
graph = {
    'A': ['B', 'D'],
    'B': ['A', 'C', 'E'],
    'C': ['B', 'E', 'F', 'G'],
    'D': ['A', 'E'],
    'E': ['B', 'C', 'D', 'G'],
    'F': ['C', 'H', 'I'],
    'G': ['C', 'E', 'I'],
    'H': ['F', 'I'],
    'I': ['F', 'G']
}

# Display the graph
for node, neighbors in graph.items():
    print(f"{node}: {neighbors}")
```

Understanding the Required Variables in DFS

1 Stack

The stack stores nodes that are discovered but not yet processed. It follows **LIFO (Last In, First Out)**.

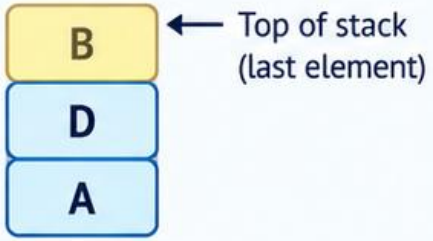
Initialization

```
stack = [start]
```

Main operations

```
stack.append(node)   # push
stack.pop()          # pop
```

Example (during execution)



Stack (bottom → top)

[A, D, B]

2 Visited Set

The **visited set** keeps track of nodes that have already been **discovered and added to the stack**. It prevents the same node from being added multiple times.

Initialization

```
visited = {start}
```

How it is used

```
if neighbor not in visited:
    visited.add(neighbor)
    stack.append(neighbor)
```

Example (during execution)

{ **A** , **B** , **C** }

These nodes have been discovered and are already in the stack or have been processed.

3 Parent Dictionary

The parent dictionary stores the predecessor (parent) of each node. It is used to **reconstruct the path** from the start node to a goal node after the search is complete.

Initialization

```
parent = {start: None}
```

How it is used

```
parent[neighbor] = current
# when a new node is discovered
# and pushed to the stack
```

Example (during execution)

Node	Parent
A	None
B	A
C	B
E	B

This shows which node first discovered each node.

4 Traversal List

The traversal list records the order in which nodes are actually **processed** (popped from the stack).

Initialization

```
traversal = []
```

How it is used

```
current = stack.pop()
traversal.append(current)
```

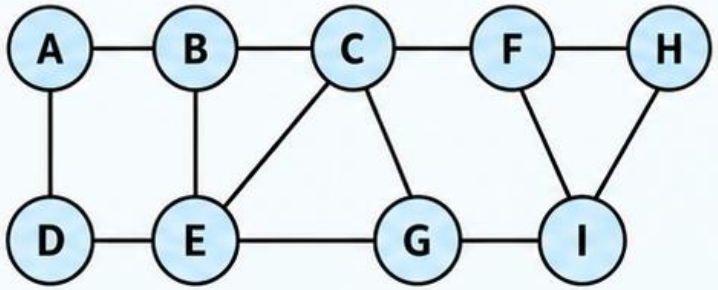
Example (during execution)

A , **B** , **C** , **F** , **H**

This list shows the DFS traversal order as nodes are processed one by one.

Complete DFS Implementation in Python (with Goal Search)

1 Graph (Visual Representation)



2 Graph (Python Representation)

```
graph = {
    'A': ['B', 'D'],
    'B': ['A', 'C', 'E'],
    'C': ['B', 'E', 'F', 'G'],
    'D': ['A', 'E'],
    'E': ['B', 'C', 'D', 'G'],
    'F': ['C', 'H', 'I'],
    'G': ['C', 'E', 'I'],
    'H': ['F', 'I'],
    'I': ['F', 'G', 'H']
}
```

Note:

- This is an **undirected** graph.
- Each connection appears in both nodes' neighbor lists.
- The order of neighbors determines which branch DFS explores first.

3 DFS Function (Returns Traversal and Path)

```
def dfs(graph, start, goal):
    # Initialize stack, visited set, parent dict and traversal list
    stack = [start]
    visited = {start}
    parent = {start: None}
    traversal = []

    # Perform DFS
    while stack:
        # Pop the top node
        current = stack.pop()
        # Add to traversal order
        traversal.append(current)
        # If goal is found, break
        if current == goal:
            break

        # Explore neighbors (in reverse order to follow given order)
        for neighbor in reversed(graph[current]):
            if neighbor not in visited:
                visited.add(neighbor)          # mark visited on push
                parent[neighbor] = current     # store parent
                stack.append(neighbor)         # push to stack

    # Reconstruct path if goal is reached
    path = []
    if goal in visited:
        node = goal
        while node is not None:
            path.append(node)
            node = parent[node]
        path.reverse()
    return traversal, path
```

4 Run the Function

```
start = 'A'
goal = 'I'

traversal, path = dfs(graph, start, goal)

print("DFS Traversal Order:", traversal)
print("Path from", start, "to", goal, ":", path)
```

5 Output

```
DFS Traversal Order:
['A', 'B', 'C', 'F', 'H', 'I', 'G', 'E', 'D']

Path from A to I :
['A', 'B', 'C', 'F', 'I']
```

6 Explanation

1. The function accepts graph, start and goal.
2. A stack is used for DFS (LIFO).
3. The **visited set** keeps track of discovered nodes and prevents duplicates (mark on push).
4. The **parent dictionary** stores the predecessor of each node to reconstruct the path.
5. The **traversal list** records the order in which nodes are processed (popped from the stack).
6. If the goal is found, the loop breaks and the path is reconstructed using the parent dictionary.